

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Guía de la Tercera Entrega del Proyecto Fin de Curso (PFC)

Ingeniería de Requisitos — Cuarto nivel

Período Académico Presencial 2026-2027

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

`gguerrero@uteq.edu.ec`

Etiqueta objetivo: v0.8.0-rc

Fecha de cierre: viernes de la semana 12

Continúa desde: Entrega 2 (v0.5.0, semana 8)

Precede a: Entrega Final (v1.0.0, semana 16)

Referencia de calidad: IngReq-Modelo.pdf secciones 5.5, 5.6 y Anexo E

Quevedo — Los Ríos — Ecuador

2026

Índice

1. Naturaleza y ubicación temporal de la Tercera Entrega	3
2. Bloque A — Modelado UML	3
2.1. Herramienta y formato exigido	3
2.2. Diagrama de casos de uso	3
2.3. Diagrama de clases (nivel de análisis)	4
2.4. Diagramas de actividad	4
2.5. Diagrama de secuencia	4
2.6. Diagrama de componentes	4
2.7. Diagrama de despliegue	5
3. Bloque B — Diagramas de análisis de requisitos	5
3.1. Diagrama de contexto	5
3.2. Diagrama de dependencia estratégica (marco i*)	5
3.3. Diagrama de requisitos SysML	6
4. Bloque C — Validación de requisitos con <i>stakeholders</i>	6
4.1. Revisión de requisitos (<i>Requirement Reviews</i>)	6
4.2. Prototipos de baja fidelidad	7
4.3. Actualización de priorización Kano	7
5. Bloque D — Documento académico integrado v0.8.0-rc	7
6. Estructura del repositorio al cierre de la Tercera Entrega	8
7. Rúbrica de evaluación de la Tercera Entrega	9
8. Procedimiento de entrega y defensa oral parcial	10
9. Aviso sobre la Entrega Final	11

1. Naturaleza y ubicación temporal de la Tercera Entrega

La Tercera Entrega es el hito de modelado y validación intermedia del PFC. Convierte la especificación Volere de la Entrega 2 en modelos gráficos verificables y ejecuta una segunda ronda de validación con *stakeholders* para confirmar la pertinencia, claridad y completitud del análisis. Se cierra el viernes de la semana 12 con la etiqueta **v0.8.0-rc** (*release candidate*) sobre el repositorio público del equipo.

La entrega pivota entre dos disciplinas complementarias. Por un lado, el modelado UML (*Unified Modeling Language*) codificado por Booch, Rumbaugh y Jacobson [1] y explicado didácticamente por Fowler [2] y Larman [3], que se emplea para describir el comportamiento, la estructura y la organización lógica y física del sistema propuesto. Por otro lado, los diagramas de análisis de requisitos, que extienden UML para tratar explícitamente los objetivos y dependencias de los actores: contexto (Kossiakoff [4], Robertson y Robertson [5]), dependencia estratégica del marco *i** (Yu [6], Dardenne y van Lamsweerde [7]) y requisitos SysML (Hause [8], Weilkiens [9]).

Objetivos concretos de la Tercera Entrega

1. Producir los seis diagramas UML del proyecto modelo (casos de uso, clases, actividad, secuencia, componentes, despliegue), articulados por módulo y con calidad tipográfica.
2. Producir los tres diagramas de análisis de requisitos (contexto, dependencia estratégica *i**, requisitos SysML).
3. Redactar los casos de uso detallados en plantilla Cockburn con niveles 1–4.
4. Ejecutar una segunda ronda de validación con *stakeholders* y actualizar la especificación donde corresponda.
5. Producir la versión **v0.8.0-rc** del documento académico integrado.

2. Bloque A — Modelado UML

2.1. Herramienta y formato exigido

El equipo utilizará *Visual Paradigm Community Edition* o *draw.io* para diseñar los diagramas. Los archivos nativos (**.vpp** o **.drawio**) deben versionarse en **assets/uml/** junto con la exportación en formato PNG con resolución mínima de 300 puntos por pulgada. Cada diagrama debe tener numeración única, título descriptivo y ser referenciado desde el texto del documento académico usando `\ref` [2, 3].

2.2. Diagrama de casos de uso

Los casos de uso describen las funcionalidades del sistema desde la perspectiva del usuario y estructuran la interacción con los actores [10, 2]. El equipo producirá al menos un diagrama de casos de uso general que abarque el sistema completo y un diagrama por módulo funcional. El proyecto modelo aporta cinco diagramas: general, módulo de gestión de disponibilidad y preferencia, módulo de gestión de solicitud de tutoría, módulo de reportes y módulo de registro y seguimiento (Figuras 2 a 6 del modelo).

Cada caso de uso identificado irá acompañado de una ficha detallada en plantilla Cockburn, con los siguientes campos: nombre, actor principal, actores secundarios, precondiciones, postcondiciones, escenario principal (numerado paso a paso), escenarios alternativos y de excepción, requisitos especiales de rendimiento o interfaz. El proyecto modelo dedica su Anexo E a estas fichas.

Entregable A.1 — Casos de uso

Archivos obligatorios:

- `assets/uml/casos-uso/` con archivos `.vpp` o `.drawio` y sus exportaciones PNG.
- `docs/arquitectura/uml/CASOS-USO.md`: descripción textual y referencia a cada diagrama.
- `docs/arquitectura/uml/CASOS-USO-DETALLADOS.tex`: fichas Cockburn compilables a PDF (mínimo doce casos de uso detallados).

2.3. Diagrama de clases (nivel de análisis)

El diagrama de clases representa las entidades del dominio, sus atributos, sus relaciones y su multiplicidad. En esta entrega el equipo produce el diagrama a *nivel de análisis*, no de diseño: no se anotan tipos primitivos de lenguaje específico ni implementaciones; el objetivo es reflejar el modelo del dominio [2, 3]. Se identificarán al menos ocho clases relevantes con al menos tres atributos cada una, y las relaciones se etiquetarán correctamente (asociación, agregación, composición, herencia, dependencia).

2.4. Diagramas de actividad

Los diagramas de actividad representan flujos de trabajo y comportamientos concurrentes. Se producirá un diagrama de actividad general y uno por módulo funcional (análogo al modelo que aporta cinco: general, gestión de disponibilidad y preferencias, reportes e integración, gestión de solicitud, registro y seguimiento). Los diagramas deben incluir *swimlanes* para separar responsabilidades de actores, y usar correctamente los nodos de decisión (rombo), de fork y de join [11, 12].

2.5. Diagrama de secuencia

El diagrama de secuencia representa las interacciones ordenadas en el tiempo entre actores y objetos del sistema. El equipo producirá al menos cuatro diagramas de secuencia correspondientes a los flujos más críticos identificados en los casos de uso (por ejemplo el flujo principal del caso de uso más usado). El proyecto modelo no reporta explícitamente diagramas de secuencia en su Sección 5.5 (por decisión de sus autores), pero esta guía sí los exige para elevar la calidad técnica del PFC del equipo.

2.6. Diagrama de componentes

El diagrama de componentes describe la organización lógica del sistema en módulos software con interfaces explícitas. Se producirá al menos un diagrama de componentes que muestre la

separación entre presentación, lógica de negocio, acceso a datos y componentes externos (por ejemplo integración con SGA, canal de notificación WhatsApp, canal de correo electrónico). El proyecto modelo aporta uno (Figura 12 del modelo).

2.7. Diagrama de despliegue

El diagrama de despliegue muestra la organización física del sistema: nodos de hardware, servidores y su conexión. Es un diagrama a nivel conceptual: no se exige nombres de proveedores ni marcas específicas. Se identificarán los nodos mínimos: cliente web, servidor de aplicaciones, servidor de base de datos, servidor de notificaciones y cualquier sistema externo integrado. El proyecto modelo aporta uno (Figura 13 del modelo).

Entregable A.2 — Diagramas UML completos

Archivos obligatorios en `assets/uml/` y sus documentaciones en `docs/arquitectura/uml/`:

- Casos de uso: general + ≥ 4 por módulo.
- Clases: al menos ocho clases con relaciones.
- Actividad: general + ≥ 4 por módulo.
- Secuencia: ≥ 4 para flujos críticos.
- Componentes: al menos uno completo.
- Despliegue: al menos uno completo.

3. Bloque B — Diagramas de análisis de requisitos

3.1. Diagrama de contexto

El diagrama de contexto define los límites del sistema y lo representa como una entidad única que interactúa con su entorno mediante flujos de datos [4, 5]. Se producirá un diagrama de contexto donde el sistema aparece como una caja central, rodeado de todos los actores externos (usuarios, sistemas externos, reguladores) y los flujos de información entre ellos y el sistema.

3.2. Diagrama de dependencia estratégica (marco i*)

El diagrama de dependencia estratégica del marco i* [6, 7] captura el “porqué” de los requisitos al modelar cómo un actor depende de otro para lograr un objetivo, completar una tarea o acceder a un recurso. Se producirá al menos un diagrama SD (*Strategic Dependency*) con todos los actores del sistema y sus dependencias mutuas.

Elementos básicos del diagrama i* SD

- **Actor** (círculo con nombre): entidad con intencionalidad, capaz de perseguir objetivos.
- **Depender / Dependee**: el actor que necesita algo (Depender) y el actor del que depende (Dependee).
- **Dependencia de objetivo** (óvalo): un actor depende de otro para lograr una meta

declarativa.

- **Dependencia de tarea** (hexágono): un actor depende de otro para ejecutar una tarea concreta.
- **Dependencia de recurso** (rectángulo): un actor depende de otro para obtener un recurso físico o de información.
- **Dependencia de *softgoal*** (óvalo con línea ondulada): objetivo cualitativo sin criterio claro de logro (por ejemplo “satisfacción del usuario”).

3.3. Diagrama de requisitos SysML

El diagrama de requisitos de SysML aborda las deficiencias de UML para el modelado explícito de requisitos: permite jerarquizar los requisitos, relacionarlos mediante *derive*, *verify*, *copy*, *trace* y *refine*, y trazarlos hasta sus fuentes y sus verificaciones [8, 9]. Se producirá al menos un diagrama SysML con los requisitos principales del sistema y sus relaciones jerárquicas. Cada requisito representado como una caja debe usar el estereotipo «**requirement**» con los campos *id* y *text*.

Entregable B.1 — Diagramas de análisis

Archivos obligatorios:

- `assets/uml/analisis/contexto.png` + fuente.
- `assets/uml/analisis/istar-sd.png` + fuente.
- `assets/uml/analisis/sysml-req.png` + fuente.
- `docs/arquitectura/analisis/README.md`: descripción textual y referencia a cada diagrama.

4. Bloque C — Validación de requisitos con *stakeholders*

La validación de requisitos garantiza que las funcionalidades propuestas están alineadas con las necesidades reales de los interesados y con los objetivos institucionales del sistema [13, 14, 15]. Una validación temprana evita errores costosos y mejora la calidad del producto final. El equipo aplicará en esta entrega tres técnicas de validación complementarias.

4.1. Revisión de requisitos (*Requirement Reviews*)

Se convocará al menos una revisión formal de requisitos con los mismos *stakeholders* entrevistados en la Entrega 2. Durante la revisión se les presentará la especificación Volere y los diagramas UML relevantes. El objetivo es verificar completitud, consistencia y claridad de los enunciados. Los hallazgos se registrarán en `docs/requisitos/validacion/REVISION-<fecha>.md` con la lista de asistentes, los hallazgos crudos, la clasificación de cada hallazgo (corrección, aclaración, adición, eliminación) y las decisiones adoptadas por el equipo.

4.2. Prototipos de baja fidelidad

Se producirán prototipos de baja fidelidad (*wireframes*) de al menos cuatro pantallas críticas identificadas en los casos de uso principales. Los prototipos se generarán en Figma o Excalidraw y se compartirán con los *stakeholders* para retroalimentación. Los archivos se versionarán en `assets/prototipos/`. El objetivo del prototipado en esta fase es discutir la viabilidad y pertinencia de las funcionalidades, no producir un diseño visual final [13, 16].

4.3. Actualización de priorización Kano

Con base en las conversaciones de validación, el equipo actualizará su tabla Kano de la Entrega 2. Cualquier reclasificación debe justificarse en `docs/requisitos/priorizacion/KANO-v0.8.md` con el nombre del *stakeholder* que motivó el cambio y la fecha de la conversación.

Entregable C.1 — Validación con *stakeholders*

Archivos obligatorios en `docs/requisitos/validacion/`:

- `REVISION-<fecha>.md`: acta de la revisión formal.
- `HALLAZGOS.md`: tabla consolidada de hallazgos con estado.
- `KANO-v0.8.md`: priorización Kano actualizada con evidencia de conversaciones.
- `assets/prototipos/`: wireframes de al menos cuatro pantallas críticas.

5. Bloque D — Documento académico integrado v0.8.0-rc

El equipo consolidará todo el trabajo realizado desde la Entrega 1 en un único documento académico compilable a PDF, con estructura IMRaD adaptada al proyecto:

1. **Resumen** (máximo 300 palabras, en español e inglés).
2. **Palabras clave** (entre cinco y ocho).
3. **Introducción**: contexto, problema, objetivos, alcance, contribución esperada.
4. **Revisión del estado del arte**: con la disciplina PRISMA 2020 (viene de la Entrega 1, con posibles actualizaciones).
5. **Sistema propuesto**: vista conceptual y módulos (Entrega 1, actualizado).
6. **Metodología**: enfoque híbrido Scrum + Kanban + Volere, roles traspolados, elicitación, validación (Entregas 1 y 2, actualizadas).
7. **Resultados y Discusión**: requisitos organizados por categoría (Entrega 2), matriz de trazabilidad, historias de usuario, priorización MoSCoW y Kano, diagramas UML, diagramas de análisis (Entrega 3).
8. **Referencias**: en formato IEEE, todas verificadas.

9. **Anexos:** instrumentos de elicitación, resúmenes de entrevistas, análisis de encuestas, requisitos Volere completos, casos de uso detallados Cockburn.

El documento se produce en LaTeX en docs/entregas/documento-academico/main.tex y se compila automáticamente en la CI de GitHub Actions. El PDF resultante (DocumentoAcademico-v0.8.0-rc.pdf) debe tener entre cincuenta y cien páginas; el proyecto modelo tiene 109 páginas, con lo cual el equipo puede tomar esa longitud como techo razonable.

Entregable D.1 — Documento académico integrado

Archivo obligatorio: docs/entregas/documento-academico/main.tex + DocumentoAcademico-v0.8.0-rc.pdf. Todas las citas deben resolver a entradas verificadas en IngReq.bib. Todas las figuras y tablas deben referenciarse en el cuerpo con \ref. Ninguna tabla ni figura debe quedar huérfana.

6. Estructura del repositorio al cierre de la Tercera Entrega

Estructura de árbol acumulada al final de la Entrega 3

```
pfc-<nombre-corto>/
+-- README.md
+-- CHANGELOG.md          (con seccion Entrega 3)
+-- CITATION.cff
+-- LICENSE
+-- docs/
|   +-- entregas/
|   |   +-- documento-academico/
|   |   |   +-- main.tex
|   |   |   +-- DocumentoAcademico-v0.8.0-rc.pdf
|   +-- requisitos/      (todo lo de las Entregas 1 y 2, actualizado)
|   |   +-- validacion/
|   |   |   +-- REVISION-<fecha>.md
|   |   |   +-- HALLAZGOS.md
|   |   |   +-- KANO-v0.8.md
|   +-- arquitectura/
|   |   +-- uml/
|   |   |   +-- CASOS-USO.md
|   |   |   +-- CASOS-USO-DETALLADOS.tex
|   |   |   +-- CLASES.md
|   |   |   +-- ACTIVIDAD.md
|   |   |   +-- SECUENCIA.md
|   |   |   +-- COMPONENTES.md
|   |   |   +-- DESPLIEGUE.md
|   |   +-- analisis/
|   |   |   +-- README.md    (contexto, i*, SysML)
|   +-- adr/
|   |   +-- ADR-001, ADR-002, ADR-003 (de Entregas anteriores)
|   |   +-- ADR-004-decisiones-uml.md
|   |   +-- ADR-005-decisiones-validacion.md
```



```

|   +-- observaciones/
|       +-- OBSERVACIONES.md
+-- assets/
|   +-- uml/
|       +-- casos-uso/, clases/, actividad/, secuencia/, componentes/, despliegue/
|       +-- analisis/      (contexto.png, istar-sd.png, sysml-req.png + fuentes)
|   +-- prototipos/
+-- scripts/
+-- .github/
|   +-- workflows/
|       +-- ci-latex.yml

```

7. Rúbrica de evaluación de la Tercera Entrega

Criterio	Peso	Excelente (100 %)	Bueno (75 %)	En desarrollo (50 %)	Insuf. (25 %)
C0. Continuidad y observaciones	5 %	Todas las OBS de Entregas 1 y 2 resueltas o diferidas con justificación.	Una o dos OBS abiertas menores.	Tres o cuatro abiertas.	Cinco o más abiertas.
C1. Diagramas de casos de uso	12 %	General + ≥ 4 por módulo + ≥ 12 fichas Cockburn detalladas.	General + 3 por módulo + ≥ 8 fichas.	General + 2 por módulo.	Solo general o menos.
C2. Diagrama de clases	8 %	≥ 8 clases con atributos y relaciones correctas.	6–7 clases correctas.	4–5 clases o relaciones incompletas.	Menos de 4 o incorrecto.
C3. Diagramas de actividad	10 %	General + ≥ 4 por módulo con <i>swimlanes</i> .	General + 3 por módulo.	General + 1–2 por módulo.	Solo general o menos.
C4. Diagramas de secuencia	8 %	≥ 4 diagramas de flujos críticos.	2–3 diagramas.	1 diagrama.	Ninguno.
C5. Diagramas de componentes y despliegue	7 %	Ambos presentes y correctos.	Ambos con detalles menores omitidos.	Uno de los dos.	Ninguno.
C6. Diagramas de análisis (contexto, i*, SysML)	15 %	Tres presentes, correctos, referenciados en el texto.	Tres presentes con detalles menores omitidos.	Dos presentes.	Uno o ninguno.

Criterio	Peso	Excelente (100 %)	Bueno (75 %)	En desarrollo (50 %)	Insuf. (25 %)
C7. Validación con <i>stakeholders</i>	15 %	Revisión formal + prototipos + Kano actualizada con evidencia.	Dos técnicas aplicadas.	Una técnica aplicada.	Ninguna aplicada.
C8. Documento académico integrado	15 %	IMRaD completo, 50–100 páginas, todas las citas verificadas, todas las figuras y tablas referenciadas.	IMRaD con secciones faltantes menores.	IMRaD con dos o más secciones ausentes.	Documento fragmentado o sin compilar.
C9. Organización del repositorio y autoría	5 %	Estructura completa, tag v0.8.0-rc, ≥ 10 commits por integrante, CI verde.	Estructura completa, autoría parcialmente desbalanceada.	Estructura parcial.	Estructura incompleta o sin tag.

Regla de calidad tipográfica

Cualquier diagrama presentado como foto de pantalla del editor con márgenes negros de fondo, escala incorrecta o texto ilegible será rechazado. El requerimiento típico es un PNG con al menos 300 DPI y con nombres de actores y objetos legibles a la escala de una página A4.

8. Procedimiento de entrega y defensa oral parcial

Cierre en repositorio con la etiqueta v0.8.0-rc. La defensa oral parcial se realiza durante la semana 13 en cuarenta minutos con estructura:

- Cinco minutos: recapitulación de Entregas 1 y 2 y observaciones resueltas.
- Diez minutos: recorrido comentado de los diagramas UML por módulo.
- Diez minutos: recorrido comentado de los diagramas de análisis (contexto, i*, SysML).
- Diez minutos: presentación de la validación y de la priorización Kano final.
- Cinco minutos: preguntas del docente y de equipos evaluadores pares.

9. Aviso sobre la Entrega Final

La Entrega Final (semana 16, v1.0.0) es la versión estable del documento académico, con métricas de calidad de requisitos INCOSE calculadas y reportadas [17], e insumos completos para publicación en revistas JCR (DOI Zenodo por separado para software y dataset, ORCID de todos los autores, CITATION.cff v1.2.0, checklist FAIR) [18, 19]. El equipo debe ir preparando desde ahora las cuentas ORCID y la organización del *deposit* en Zenodo.

Referencias

- [1] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA: Addison-Wesley Professional, 2005.
- [2] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2018.
- [3] C. Larman, *UML y Patrones: Una Introducción al Análisis y Diseño Orientado a Objetos y al Proceso Unificado*, 2nd ed. Madrid, España: Pearson Educación, 2003.
- [4] A. Kossiakoff, W. N. Sweet, S. J. Seymour, and S. M. Biemer, *Systems Engineering: Principles and Practice*, 2nd ed., ser. Wiley Series in Systems Engineering and Management. Hoboken, NJ: Wiley-Interscience, 2011.
- [5] S. Robertson and J. Robertson, *Mastering the Requirements Process: Getting Requirements Right*, 3rd ed. Boston, MA: Addison-Wesley Professional, 2012.
- [6] E. S. K. Yu, “Towards Modelling and Reasoning Support for Early-Phase Requirements Engineering,” in *Proceedings of the 3rd IEEE International Symposium on Requirements Engineering (RE’97)*, 1997, pp. 226–235.
- [7] A. Dardenne, A. van Lamsweerde, and S. Fickas, “Goal-Directed Requirements Acquisition,” *Science of Computer Programming*, vol. 20, no. 1–2, pp. 3–50, Apr. 1993.
- [8] M. Hause, “The SysML Modelling Language,” in *Fifth European Systems Engineering Conference (EuSEC)*, Edinburgh, UK, Sep. 2006.
- [9] T. Weillkiens, *Systems Engineering with SysML/UML: Modeling, Analysis, Design*. Burlington, MA: Morgan Kaufmann / Elsevier, 2011.
- [10] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley Professional, 2001.
- [11] T. Ahmad, J. Iqbal, A. Ashraf, D. Truscan, and I. Porres, “Model-Based Testing Using UML Activity Diagrams: A Systematic Mapping Study,” *Computer Science Review*, vol. 33, pp. 98–112, Aug. 2019.
- [12] B. Paech, “On the Role of Activity Diagrams in UML: A User Task Centered Development Process for UML,” in *The Unified Modeling Language — UML’98: Beyond the Notation*,

- ser. Lecture Notes in Computer Science, vol. 1618. Springer, Berlin, Heidelberg, 1999, pp. 267–277.
- [13] I. Sommerville, *Software Engineering*, 9th ed. Boston, MA: Addison-Wesley, 2011.
 - [14] K. E. Wieggers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA: Microsoft Press, 2013.
 - [15] J. Dick, E. Hull, and K. Jackson, *Requirements Engineering*, 4th ed. Cham, Switzerland: Springer International, 2017.
 - [16] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Heidelberg: Springer, 2010.
 - [17] INCOSE Requirements Working Group, “Guide to Writing Requirements,” International Council on Systems Engineering, San Diego, CA, INCOSE Technical Product INCOSE-TP-2010-006-04, Jul. 2023.
 - [18] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg *et al.*, “The FAIR Guiding Principles for Scientific Data Management and Stewardship,” *Scientific Data*, vol. 3, p. 160018, 2016.
 - [19] P. Ralph, N. bin Ali, S. Baltes *et al.*, “Empirical Standards for Software Engineering Research,” *ACM SIGSOFT Software Engineering Notes*, vol. 46, no. 3, pp. 19–20, 2021.